

Dungeon Draft — Guide de création des maps & design des salles

Document de référence pour construire les salles du jeu. Deux parties : **(A)** le setup technique du TileMap (pour celui qui dessine), **(B)** la méthode de design des salles (pour vous deux).

Rappel d'architecture (à garder en tête en permanence)

Le terrain du jeu vit sur **trois couches**, chacune avec un rôle précis :

Couche	Rôle	Quand elle agit
TileMap (TerrainLayer)	Là où on dessine le terrain statique	Lu une seule fois au démarrage du combat
GridData	La vérité unique du terrain pendant le combat	En permanence (pathfinding, sorts, IA lisent ici)
grid_view	L' affichage de la vérité logique (highlights)	En permanence (dessin)

Règle d'or : GridData est le seul patron pendant le combat. Le TileMap n'est qu'une **source d'entrée** au démarrage et un **décor** à l'arrivée. Il n'est **jamais** un acteur de la logique.

Conséquence : on peut changer tout l'habillage visuel (rectangles → TileMap → sprites isométriques un jour) sans jamais toucher aux règles du jeu.

PARTIE A — Setup technique du TileMap

Pour la personne qui dessine les maps. À faire une fois pour le TileSet, puis répétable pour chaque salle.

A.1 — Préparer les tiles

- Les tiles doivent faire **64 × 64 pixels** (la grille logique utilise CELL_SIZE = 64). Si les tiles font 32×32, mettre le TileMapLayer à scale = (2, 2) — mais le plus simple est de dessiner directement en 64.
- Importer l'image dans le projet, créer un **TileSet**, ajouter les tiles en source atlas. Pour démarrer : une tile de sol, une tile de mur suffisent.

A.2 — Ajouter le custom data layer `cell_type` (étape clé)

C'est l'étiquette invisible qui dit à chaque tile **ce qu'elle est au sens gameplay** (indépendamment de son apparence).

1. Sélectionner la ressource **TileSet** (pas le nœud `TileMapLayer`).
2. Dans l'inspecteur, déplier **Custom Data Layers**.
3. Ajouter un élément :
 - **Name** : `cell_type`
 - **Type** : `String`

Un seul layer suffit pour tout le terrain.

A.3 — Étiqueter chaque tile

1. Ouvrir l'éditeur de `TileSet` (panneau en bas), onglet **Select**.
2. Cliquer sur une tile.
3. Dans l'inspecteur, remplir le champ `cell_type` apparu.

Valeurs autorisées (en MAJUSCULES, exactement) :

<code>cell_type</code>	Sens	Marchable	Bloque la vue
<code>NORMAL</code>	Sol standard	Oui	Non
<code>WALL</code>	Mur	Non	Oui
<code>HOLE</code>	Trou / vide	Non	Non (on voit par-dessus)
<code>LAVA</code>	Lave	Oui	Non
<code>ICE</code>	Glace	Oui	Non
<code>SHADOW</code>	Ombre / brouillard	Oui	Oui (bloque la vue)
<code>RUNE</code>	Rune magique	Oui	Non

Notes importantes :

- L'**apparence ne compte pas**, seule l'étiquette définit le gameplay. 5 variantes visuelles de mur → toutes étiquetées `WALL`.
- Une tile sans `cell_type` (ou avec un texte inconnu) sera traitée comme `NORMAL` par sécurité.
- **Pour démarrer, n'utiliser que `NORMAL` et `WALL`**. Les autres types ne seront étiquetés que lorsque leurs mécaniques seront activées dans le jeu, pour éviter de dessiner du

terrain qui ne fait rien encore.

A.4 — Placer le TileMapLayer dans la scène

1. Ouvrir `battle.tscn`.
2. Ajouter un nœud `TileMapLayer` (pas l'ancien `TileMap`, déprécié).
3. Le nommer **exactement** `TerrainLayer` (c'est ce nom que le code cherche).
4. En faire un **enfant du nœud** `Battle`.
5. Lui assigner le TileSet.
6. Dessiner la salle dedans.

A.5 — Vérifier que l'import marche

1. Dessiner quelques murs dans `TerrainLayer`.
2. Lancer le jeu.
3. Vérifier que les unités **contournent** les murs (ne les traversent pas, ne s'arrêtent pas dessus).

Si le pathfinding contourne les murs → l'import a fonctionné, `GridData` a bien reçu les `WALL`. Si les unités traversent → vérifier le nom `TerrainLayer`, l'étiquette `cell_type`, et que les tiles sont bien à la position attendue.

A.6 — Attention : superposition visuelle temporaire

Tant que le développement n'est pas avancé, `grid_view` continue de dessiner ses **rectangles colorés** par-dessus le TileMap. On verra donc le décor **et** les carrés en même temps. C'est normal pour l'instant.

Quand le TileMap deviendra le vrai décor visuel, on désactivera le dessin des types de cases dans `grid_view` (il ne gardera que les **surbrillances** : déplacement, portée de sort). À faire **seulement** quand le décor TileMap est validé — sinon on ne verrait plus les murs du tout.

PARTIE B — Méthode de design des salles

┆ Pour vous deux. Comment penser une salle pour qu'elle soit intéressante à jouer, pas juste un rectangle vide.

B.1 — Les deux fichiers d'une salle

Une salle = **deux choses complémentaires** :

1. **Le terrain** (le TileMap `TerrainLayer`) → murs, trous, terrain spécial. C'est le *décor jouable*.

2. Le `RoomData.tres` → quels ennemis, zones de spawn héros et ennemis. C'est le *peuplement*.

Les deux ensemble font la salle. Le terrain pose le **terrain de jeu**, le `RoomData` pose **qui se bat dessus et où**.

B.2 — Les zones de déploiement (le cœur du design tactique)

Dans le `RoomData`, deux pools de cases :

- `hero_spawn_zone` : où le joueur pose (à terme : choisit de poser) ses héros. À placer d'un côté de la map.
- `enemy_spawn_zone` : où les ennemis apparaissent (aléatoirement). À placer de l'autre côté.

Règle pratique : toujours mettre **plus de cases dans la zone que d'ennemis**. Sinon le placement "aléatoire" n'a aucun choix. Exemple : 7 cases pour 5 ennemis → chaque run varie. 5 cases pour 5 ennemis → toujours le même placement.

Distance entre zones : éloigner les deux zones crée une phase d'approche (les unités doivent avancer). Les rapprocher crée un combat immédiat et brutal. C'est un levier de rythme : varier d'une salle à l'autre.

B.3 — Principes de level design tactique

Une bonne salle de tactical n'est pas un terrain plat. Quelques leviers :

Les murs créent des décisions. Un mur force à contourner (coûte des PM), bloque la ligne de vue (protège des sorts à distance), crée des goulots d'étranglement (un point de passage que les ennemis doivent emprunter). Un couloir étroit avantage une équipe défensive ; un espace ouvert avantage la mobilité et les sorts de zone.

La ligne de vue est une ressource. Les sorts à distance ont besoin de voir leur cible (`needs_line_of_sight`). Placer des murs, c'est créer des angles morts et des positions de couverture. Un mage embusqué derrière un mur est en sécurité ; pour l'atteindre, il faut le contourner.

Le terrain spécial crée des zones de risque/récompense (plus tard). Quand la lave/glace seront actives : une case de lave au centre est un raccourci dangereux. Un pont étroit au-dessus d'un trou force un choix. Le terrain dangereux récompense le joueur qui pousse les ennemis dessus (synergie avec les sorts de déplacement, à venir).

L'asymétrie est intéressante. Une salle parfaitement symétrique est ennuyeuse. Décaler les zones, mettre un obstacle d'un seul côté, créer un avantage de terrain à conquérir : ça donne des décisions.

B.4 — Le rythme sur un run (l'enchaînement des salles)

Les salles ne se conçoivent pas isolément mais comme une **courbe**. Sur un run de plusieurs

salles, viser une progression :

- **Salle 1** : simple. Peu d'ennemis, terrain ouvert. On prend ses marques.
- **Salles du milieu** : introduire un élément à la fois (un goulot, un terrain spécial, un type d'ennemi différent). Une idée par salle.
- **Salle finale / boss** : combiner les éléments vus avant. Le terrain devient un acteur à part entière.

Comme il n'y a **pas de regen entre les salles**, la difficulté monte naturellement : les héros arrivent entamés. Le design doit en tenir compte — une salle tardive avec beaucoup d'ennemis sur des héros à 40% de PV est brutale. C'est voulu, mais ça se dose.

B.5 — Checklist avant de valider une salle

- ☐ Le terrain est dessiné dans `TerrainLayer` (tiles étiquetées `cell_type`)
 - ☐ Un `RoomData.tres` existe pour la salle
 - ☐ `enemies` rempli (chaque entrée pointe vers un `UnitData.tres` valide)
 - ☐ `hero_spawn_zone` : assez de cases pour tous les héros, d'un côté
 - ☐ `enemy_spawn_zone` : **plus** de cases que d'ennemis, de l'autre côté
 - ☐ La salle est ajoutée à `ROOM_PATHS` dans `game_manager.gd`
 - ☐ Test : les unités contournent les murs (import OK)
 - ☐ Test : le combat est gagnable mais pas trivial
 - ☐ La salle apporte **une** idée nouvelle par rapport à la précédente
-

Récapitulatif : "données → formulaire, comportement → code"

Pour le projet en général, garder cette règle de répartition :

- **Données** (un sort, une unité, un statut, une salle, un terrain) → fichier `.tres` éditable dans l'inspecteur, **sans coder**.
- **Comportement / règles** (comment un sort s'exécute, comment une réaction de terrain se déclenche, comment l'IA décide) → **code en dur**.

Créer du contenu = remplir des formulaires. Créer une nouvelle mécanique = toucher au code.